
Lecture 3: The Tower of Babel

Communication protocols and the joy of debugging.

Robojackets Firmware Training 2026

Administrivia - Training Outline

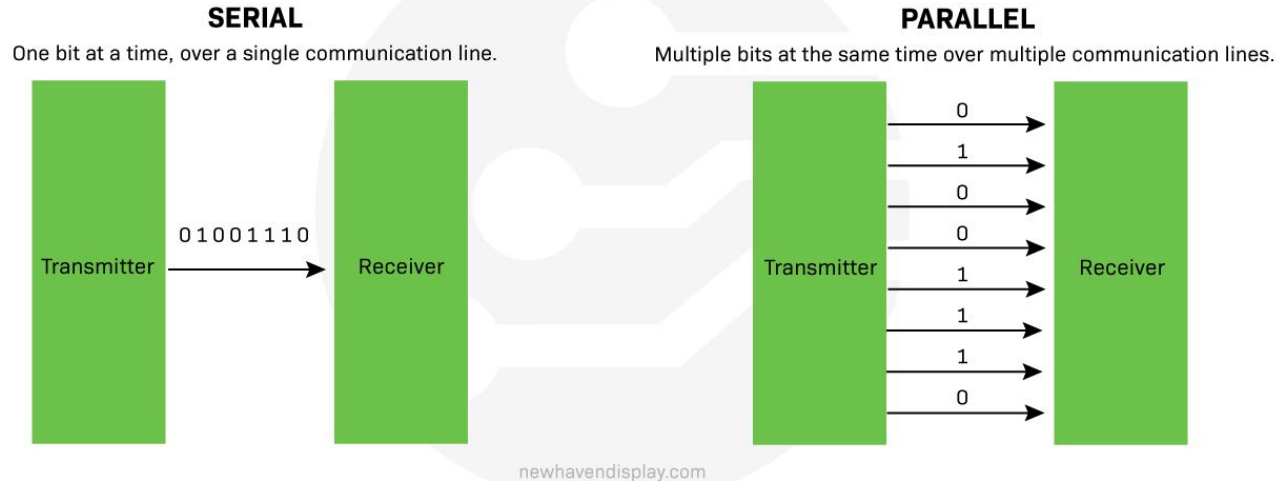
- Lecture 1: C/C++ Basics
 - Lecture 2: Interrupts & Analog I/O
 - **Lecture 3: Communication Protocols (today)**
 - Lecture 4: Driver Development
 - Lecture 5: State Machines & RTOS
 - Practical Soldering Training
-

Parallel

Parallel vs Serial

A parallel communication protocol utilizes multiple wires to send multiple bits of information at once, while a serial protocol uses one line* to send one bit at a time.

Serial vs Parallel



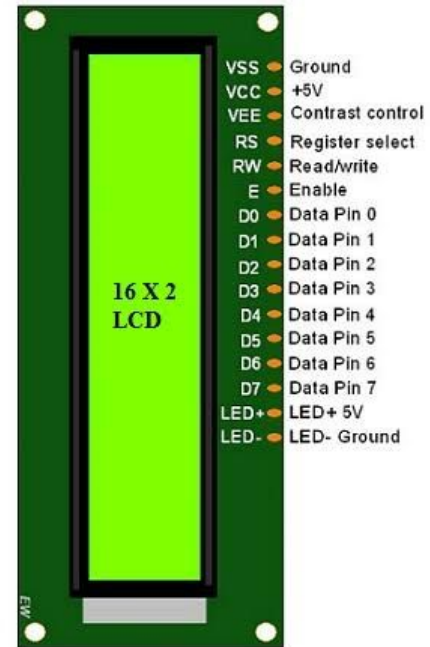
*Depending on the protocol this can still be multiple wires

Parallel Applications

- **Pros**
 - High bandwidth
 - Can be fast
 - Simple
- **Cons**
 - Lots of wires needed
 - High GPIO cost
 - Prone to crosstalk

Common Parallel Uses:

- Simple Screens
- Out of scope
 - PCI
 - IDE



Universal Asynchronous Receiver-Transmitter (UART)

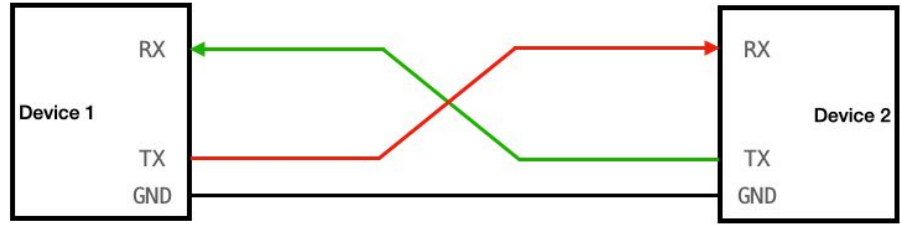
UART Format

UART transmits information between two devices across two wires in small packets. Data is sent asynchronously (not clocked) at a fixed baud rate.

Common Baud Rates:

- 9600
- 38400
- 115200 ←recommended for most
- 1500000

Only sends <9 data bits per packet. Has a parity bit for data integrity checking. This parity bit is 1/0 if the data frame is evaluated as odd/even respectively.



Tx is the “transmit” line
Rx is the “receive” line

Start Bit (1 bit)	Data Frame (5 to 9 Data Bits)	Parity Bits (0 to 1 bit)	Stop Bits (1 to 2 bits)
------------------------	------------------------------------	-------------------------------	------------------------------

UART Applications

- **Pros**
 - Simple
 - Only needs two wires
- **Cons**
 - Slow
 - One device per UART line

The most common uses of UART:

- Debugging
 - Simple peripheral control
 - Inter-microcontroller communication
-

Serial Print

```
23 #include <Arduino.h>
24
25 void setup() {
26   // Have to begin with a valid baud rate
27   Serial.begin(115200);
28   Serial.println("Hello World!");
29
30   // \n starts a new line \t adds a tab
31   Serial.print("Start\nEnd!\tAgain!");
32   // println adds \n automatically
33   Serial.println("Alone?");
34   // Serial can print strings or integers
35   Serial.println(103);
36   // Or floats
37   Serial.println(103.46732342);
38   // floats default to 2 point precision but can be changed
39   Serial.println(103.46732342, 0);
40   Serial.println(103.46732342, 8);
41   // Integers can be displayed in custom formats!
42   Serial.println(103, BIN);
43   Serial.println(103, OCT);
44   Serial.println(103, DEC);
45   Serial.println(103, HEX);
46   // Or print straight from vars
47   uint16_t num = 32102;
48   String str = "HIIII";
49   Serial.println(str);
50   Serial.println(num);
51   // For better formatting options use snprintf()
52   // Create a fixed size character buffer
53   char buf[64];
54   snprintf(buf, sizeof(buf), "Num: %d, String: %s\n", num, "hi");
55   Serial.println(buf);
56   // Be warned not all formats are allowed on all mcus
57   // The nano does not support %f, %lld, etc due to size
58 }
```

Serial printing is one of the most common debugging utilities included in modern microcontrollers. How this is implemented differs by HAL but are generally similar



```
Hello World!
Start
      End!      Again!Alone?
103
103.47
103
103.46732330
1100111
147
103
67
HIIII
32102
Num: 32102, String: hi
```

Compile Time Code Options

Compile time code allows you to exclude code from actual compilation. This can be used to choose whether to include features, save memory, or for libraries (we will go over this next week).

Common functions include:

- `#define <name> <val>`:
 - Define creates a compiler side variable with a name and value.
 - “val” is optional if it is just being used for `#ifdef`
- `#ifdef <name>`
 - Only includes code contained in it if “name” is defined at some point prior.
 - Has to be closed with `#endif`
- `#if <bool>`
 - Only includes code contained in it if “bool” is evaluated as true.
 - Has to be closed with `#endif`
- `#pragma once`
 - Ensures a library is only imported once
 - In C you use a `#def/#ifdef` header instead

```
1  #include <Arduino.h>
2
3  // #define creates a compile side variable
4  // This can either be just a name
5  #define DEBUG
6  // Or a real variable
7  #define DEBUG_VAL 1
8  #define LED_PIN 13
9
10 bool led_state = true;
11
12 void setup() {
13     // #ifdef allows code to only be included if a name is defined
14     #ifdef DEBUG
15         Serial.begin(115200);
16         Serial.println("Hello World!");
17     #endif
18
19     pinMode(LED_PIN, OUTPUT);
20 }
21
22 void loop() {
23     digitalWrite(LED_PIN, led_state);
24     // #if allows code to only be included if a value is true
25     #if DEBUG_VAL
26         Serial.print("LED state: ");
27         // Ternary operator! (boolean expression ? if true : if false)
28         Serial.println((led_state ? "ON" : "OFF"));
29     #endif
30     led_state = !led_state;
31     delay(500);
32 }
```

Serial Peripheral Interface (SPI)

SPI Format

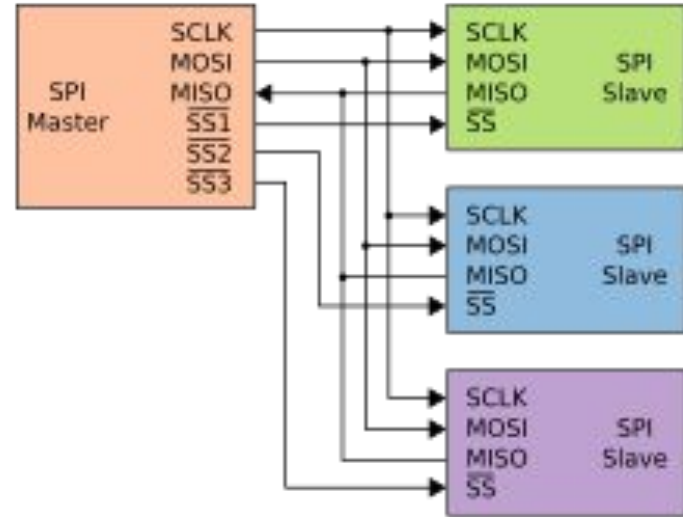
SPI is a synchronous serial communication method. SPI has one control device (master) and any number of controlled devices (slaves). SPI is a duplex communication method, where the slave sends a bit when the master sends one.

Shared Wires:

- SCLK (CLK, SCL)
 - Clock line
- MOSI
 - Master Out Slave In (Master Tx)
- MISO
 - Master In Slave Out (Master Rx)

Seperate Wires

- SS (CS, CE)
 - Slave Select
 - One per slave device

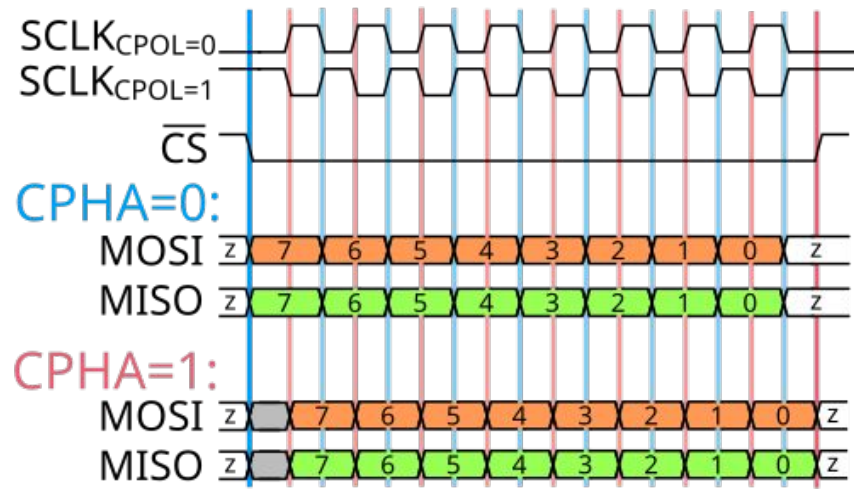


SPI Modes

All devices must be configured to the same mode, bit order, and frequency for proper communication.

While the MCU is easily configured, slave devices are often preconfigured will need to have their formats matched.

Frequency depends on devices but usually ~1-2Mhz



SPI mode	Clock polarity (CPOL)	Clock phase (CPHA)	Data is shifted out on	Data is sampled on
0	0	0	falling SCLK, and when $\overline{SS}$ activates	rising SCLK
1	0	1	rising SCLK	falling SCLK
2	1	0	rising SCLK, and when $\overline{SS}$ activates	falling SCLK
3	1	1	falling SCLK	rising SCLK

SPI Applications

- **Pros**

- Fast
- Widely used
- Customizable
 - No max clock frequency
 - Variable packet sizes
- Does not rely on addresses

- **Cons**

- Only reliable at short ranges
- Requires additional wire per device
- Non Standardized
 - Modes
- No error checking

SPI is a very widely adopted protocol, so you will find it in many many many devices.

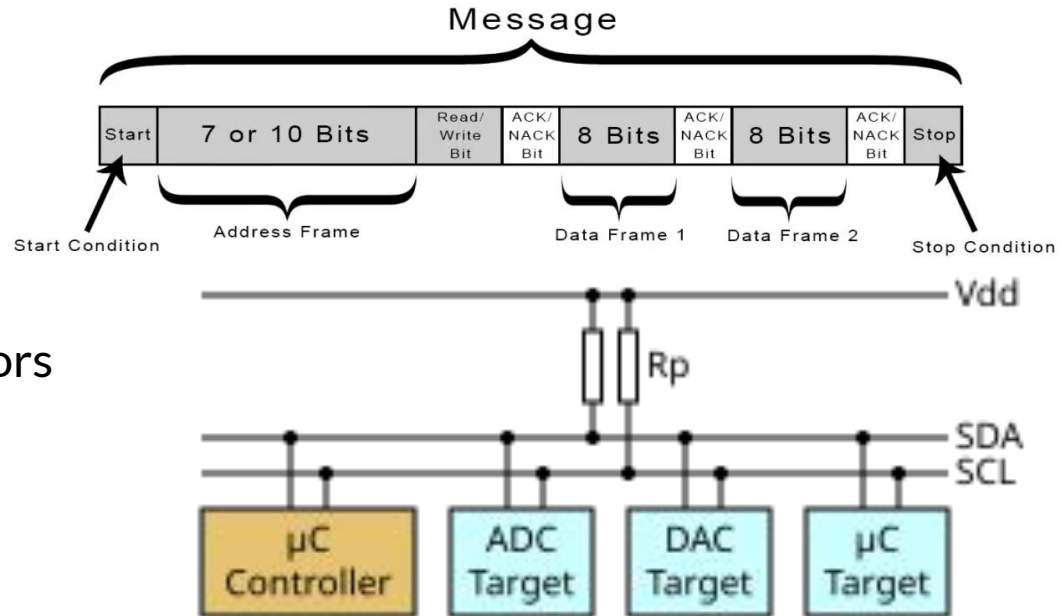
- Sensors
 - Displays
 - Memory
 - Shift Registers
-

Inter-Integrated Circuit (I2C)

I2C Format

I2C is another synchronous serial communication method that supports multiple slave devices, but also multiple masters. I2C trades driver simplicity and speed for physical simplicity and expandability. I2C devices can have 7 or 10 bit addresses, meaning up to 1008 devices can share one I2C bus*.

- SDA
 - Serial Data wire
- SCL
 - Serial Clock Line
- R
 - I2C require pullup resistors
 - Usually 4.7k ohm



I2C Applications

I2C has similar applications to SPI with notable advantages and disadvantages.

- **Pros:**
 - Only 2 wires
 - Many many devices
- **Cons:**
 - Limited by device addresses
 - Slow
 - Sensitive to line pollution

It is advised to limit the number of devices on a single I2C bus as any device can block the bus for any reason.

I²C modes

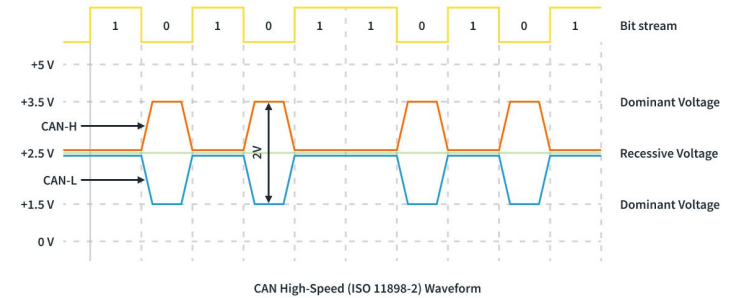
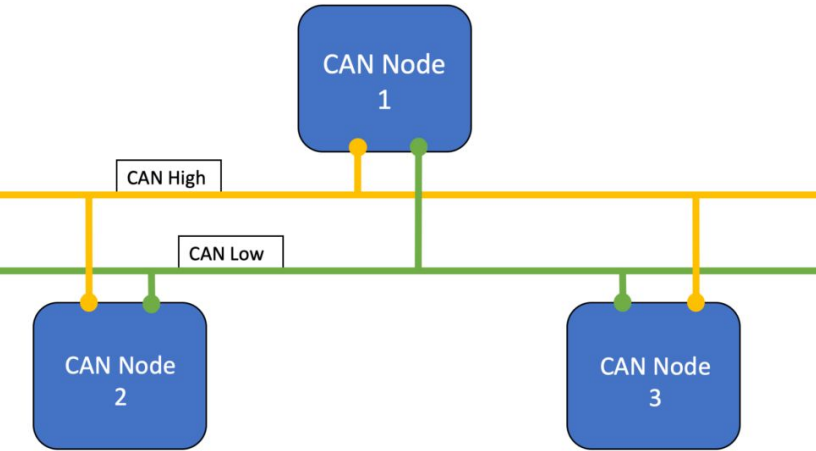
Mode ^[3]	Maximum speed	Maximum capacitance	Drive	Direction
Standard mode (Sm)	100 kbit/s	400 pF	Open drain*	Bidirectional
Fast mode (Fm)	400 kbit/s	400 pF	Open drain*	Bidirectional
Fast mode plus (Fm+)	1 Mbit/s	550 pF	Open drain*	Bidirectional
High-speed mode (Hs)	1.7 Mbit/s	400 pF	Open drain*	Bidirectional
High-speed mode (Hs)	3.4 Mbit/s	100 pF	Open drain*	Bidirectional
Ultra-fast mode (UFm)	5 Mbit/s	?	Push-pull	Unidirectional

Controller Area Network (CAN)

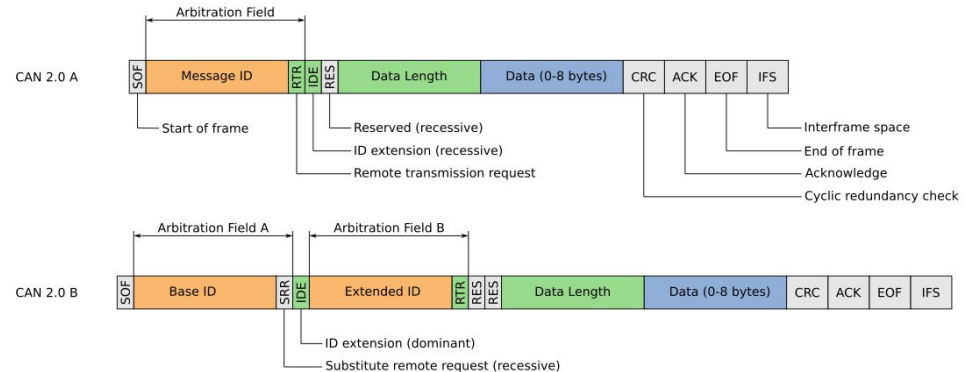
How

CAN is an asynchronous serial communication protocol that is similar to I2C. Also using just two wires, however in this case for differential pairs.

Arbitration uses a priority system rather than a first-come system.



CAN High-Speed (ISO 11898-2) Waveform

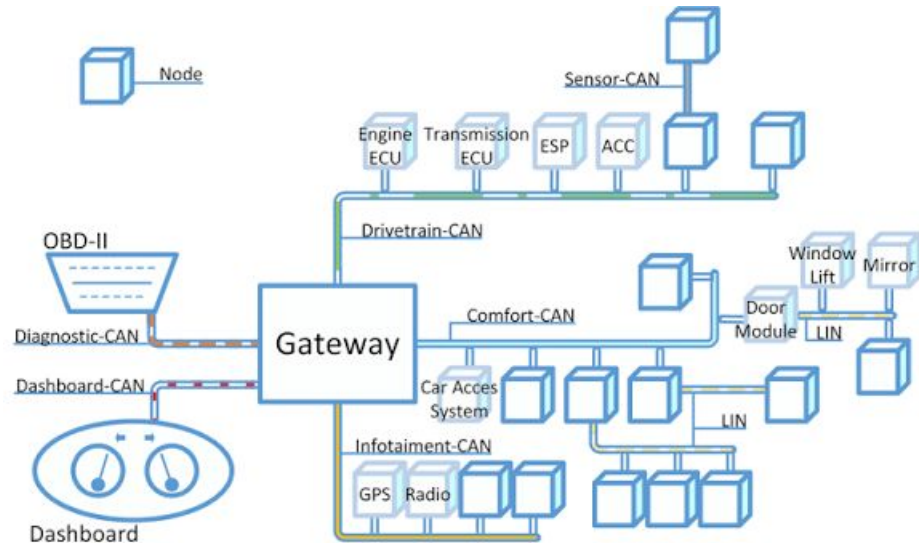


CAN Applications

- **Pros:**
 - 2 Wires
 - Reliable
 - Scalable
- **Cons:**
 - Slow
 - Limited range

CAN bus is primarily used for:

- Automotive
- Industrial Lines
- Some Robotics



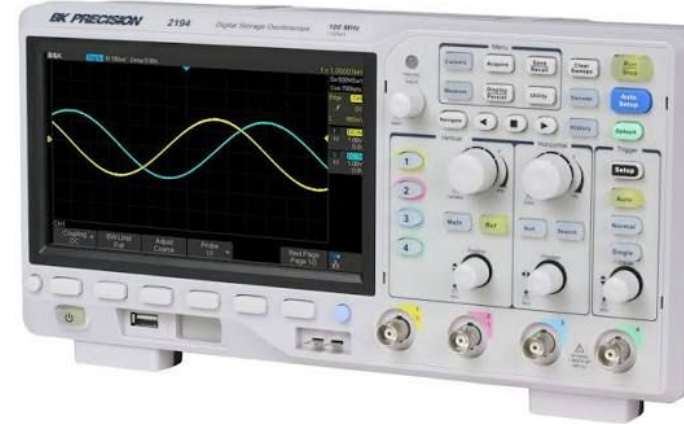
External Debugging

Oscilloscope

Oscilloscopes can be used to plot small, high frequency changes in voltage, or large low frequency, or digital logic levels, etc etc.

Oscilloscopes can measure almost anything to do with voltages and signals.

However, oscilloscopes are *VERY* expensive, and rather large. Therefore they are not always the best solution.



Logic Analyzer

Logic analyzers can be used to debug any logic level change implementations. This includes communication protocols.

While an oscilloscope can technically do the same, logic analyzers are much cheaper, and are more purposefully built for things like this.



LAB TIME!!!

Labs:

- **Serial Control:**

- Establish serial communication with your laptop and use it to control the Nano
- You should be able to set the brightness of each LED from your laptop

- **I2C IMU:**

- Use I2C to interface with the IMU breakout board
 - Read values from the IMU to detect shaking and toggle LEDs accordingly
-